

Projeto Final C213 - Esteira Inteligente com Controle Fuzzy

Documentação Técnica: Código Comentado e Conformidade

INATEL - C213 - 2026

Sumário

Sumário	1
1. Conformidade com os requisitos do professor	2
2. Código comentado (linha a linha por blocos)	3
Linhas 1–24	3
Linhas 26–46	3
Linhas 49–56	4
Linhas 58–59	4
Linhas 61–86	4
Linhas 88–110	5
Linhas 112–131	5
Linhas 133–147	5
Linhas 149–205	6
Linhas 207–225	6
Linhas 227–260	7
Linhas 262–285	7
Linhas 287–309	8

1. Conformidade com os requisitos do professor

Requisito	Onde / como
Modelar planta do zero	Esteira: modelo dinâmico de 1a ordem (linha 58-59, 133)
Tema diferente dos vistos em sala	Esteira ($\neq$ umidade do solo, temperatura de sala, persianas)
Variável controlada / manipulada / perturbação	RPM / PWM / carga
Fuzzificação + base de regras + defuzzificação	Pertinências (88-110), regras 5x5 e centroide (112-131)
Rodar na ESP32 em C/C++	sketch.ino
Interação física: display com SP/métrica	LCD I2C (245-251)
Perturbação por potenciômetro	Pot sempre ativo (135-141)
MQTT com dashboard em nuvem	HiveMQ + Node-RED
3 set points no dashboard	Botões SP1/SP2/SP3
Gráfico em tempo real	RPM x tempo
Dashboard: gráfico das funções de pertinência	Grupo Logica Fuzzy
Dashboard: tabela de regras	Grupo Logica Fuzzy
Lógica coerente com a planta	Esteira lenta -> acelerar; rápida -> frear (regras antissimétricas)

2. Código comentado (linha a linha por blocos)

Cada bloco abaixo traz as linhas numeradas seguidas de uma explicação. As quatro etapas do fuzzy (fuzzificação, regras, inferência, defuzzificação) estão sinalizadas.

Linhas 1–24

Cabeçalho e includes. Bibliotecas: WiFi/WiFiClientSecure (conexão TLS), PubSubClient (MQTT), Wire+LiquidCrystal_I2C (display) e ESP32Servo (atuador).

```
1 // =====
2 // PROJETO FINAL C213 - Esteira Inteligente com Controle Fuzzy
3 // Versao CONSOLIDADA (reaproveita o mapeamento de pinos do grupo)
4 // Grupo: [NOME 1] / [NOME 2] / [NOME 3] - INATEL 2025
5 // =====
6 // ESTE ARQUIVO UNIFICA:
7 // - a base do grupo (servo no D18, LEDs 25/26/27, pot D34, I2C 21/22)
8 // - + buzzer, HC-SR04 e botao de emergencia (novos, em pinos livres)
9 // - + controlador FUZZY REAL (pertinencia + centroide)
10 // - + alarme sonoro no buzzer (corrige o ponto 1)
11 // - + travamento so apos 4 min (corrige o ponto 2)
12 // - + MQTT/HiveMQ p/ dashboard Node-RED (entregavel obrigatorio, 15 pts)
13 // -----
14 // DISPLAY: este codigo usa LCD I2C 16x2 (como na sua foto).
15 // Se voces usarem o OLED SSD1306, ver o bloco comentado "OPCAO OLED".
16 // SEGURANCA: nao versionar a senha. Use secrets.h no .gitignore.
17 // =====
18
19 #include <WiFi.h>
20 #include <WiFiClientSecure.h>
21 #include <PubSubClient.h>
22 #include <Wire.h>
23 #include <LiquidCrystal_I2C.h>
24 #include <ESP32Servo.h>
```

Linhas 26–46

Mapeamento de pinos e credenciais MQTT. A senha fica como placeholder e deve vir do secrets.h.

```
26 // ----- PINOS (reaproveitados + novos) -----
27 #define POT_PIN      34    // (reuso) potenciometro -> carga
28 #define MOTOR_PIN    18    // (reuso) atuador: servo OU driver de motor CC
29 #define LED_VERDE     25    // (reuso)
30 #define LED_AMARELO   26    // (reuso)
31 #define LED_VERMELHO  27    // (reuso)
32 // I2C do display: SDA=21, SCL=22 (reuso)
33 #define BUZZER_PIN    32    // (NOVO) alarme sonoro
34 #define TRIG_PIN      5     // (NOVO) HC-SR04
35 #define ECHO_PIN      23    // (NOVO) HC-SR04
36 #define BTN_EMERG     33    // (NOVO) botao de emergencia (pull-up)
37
38 LiquidCrystal_I2C lcd(0x27, 16, 2);
39 Servo motor; // atuador (visual no Wokwi). Em HW real: motor CC + driver.
40
41 // ----- WiFi / MQTT (HiveMQ) -----
42 const char* ssid      = "Wokwi-GUEST";
43 const char* password  = "";
44 const char* mqtt_host = "b6c228153f69435486ef56fc32db30fc.s1.eu.hivemq.cloud";
45 const int   mqtt_port = 8883;
46 const char* mqtt_user = "Projeto-C213";
```

Linhas 49–56

Tópicos MQTT (publicação e comando) e instâncias do cliente seguro e do PubSubClient.

```
49 const char* TOP_RPM="esteira/rpm", *TOP_PWM="esteira/pwm", *TOP_ERRO="esteira/erro",
50             *TOP_CARGA="esteira/carga", *TOP_SP="esteira/setpoint", *TOP_PRODUTOS="esteira/produtos",
51             *TOP_TAMANHO="esteira/tamanho", *TOP_THRU="esteira/throughput", *TOP_STATUS="esteira/status",
52             *TOP_ALERTA="esteira/alerta", *TOP_CMD_SP="esteira/cmd/setpoint",
53             *TOP_CMD_LOAD="esteira/cmd/carga", *TOP_CMD_EMERG="esteira/cmd/emergencia";
54
55 WiFiClientSecure espClient;
56 PubSubClient client(espClient);
```

Linhas 58–59

Parâmetros da PLANTA: K (ganho do motor), B (atrito), C (efeito da carga) e dt (passo de 0,1 s). Calibrados para resposta realista.

```
58 // ----- PLANTA (mesmos parametros do grupo) -----
59 const float K=1.2, B=0.08, C=0.5, dt=0.1;
```

Linhas 61–86

Variáveis de estado + parâmetros das novas funções: contagem híbrida, carga atrelada ao produto (PESO_P/M/G e DECAIMENTO) e os dois tons da sirene.

```
61 // ----- VARIÁVEIS -----
62 float setpoint=500, carga=0, rpm=0, pwm=30, erro=0, erroAnt=0, deltaErro=0;
63 bool  emergencia=false;
64 int   contadorProdutos=0; float ultimoTamanho=0; bool produtoPresente=false;
65 unsigned long tProdutos[64]; int idxProd=0;
66 unsigned long tBloqueioInicio=0; // marca quando o sensor começou a ficar bloqueado
67
68 // >>> PONTO 2: travamento so apos 4 minutos de bloqueio continuo <<<
69 const unsigned long JAM_TIMEOUT_MS = 4UL*60UL*1000UL; // 4 minutos
70
71 // >>> CONTAGEM HÍBRIDA: automatica por tempo + sensor manual <<<
72 bool modoAuto = true; // liga a contagem automatica
73 const unsigned long INTERVALO_AUTO = 3000; // uma caixa a cada 3 s (ajustavel)
74 unsigned long tProxAuto = 0; // proximo instante de contagem auto
75 String classeTam = "-"; // Pequeno / Medio / Grande
76
77 // >>> CARGA ATRELADA AO PRODUTO <<<
78 // Cada caixa que entra soma um peso conforme o tamanho (P/M/G); essa parcela
79 // decai com o tempo (caixas saindo da esteira). O potenciometro continua
80 // valendo como carga-base (sempre ativo) e e' somado a essa parcela.
81 float cargaProdutos = 0.0f; // parcela de carga vinda das caixas
82 const float PESO_P = 3.0f, PESO_M = 6.0f, PESO_G = 10.0f; // kg por classe
83 const float DECAIMENTO = 0.3f; // kg/ciclo que "sai" da esteira
```

```

84
85 // >>> BUZZER: sirene de dois tons <<<
86 const int TOM_A = 880, TOM_B = 1320; // Hz alternados

```

Linhas 88–110

Funções de pertinência (triangular e trapezoidal) e definição dos 5 conjuntos do Erro (± 500) e do dErro (± 40). É a FUZZIFICAÇÃO.

```

88 // =====
89 // CONTROLADOR FUZZY REAL (pertinencia + min + centroide)
90 // -> corrige o ponto 4: agora ha grau de pertinencia e interpolacao
91 // =====
92 float tri(float x,float a,float b,float c){
93     if(x<=a||x>=c) return 0; if(x==b) return 1;
94     return (x<b)?(x-a)/(b-a):(c-x)/(c-b);
95 }
96 float trapEsq(float x,float b,float c){ if(x<=b)return 1; if(x>=c)return 0; return (c-x)/(c-b);}
97 float trapDir(float x,float b,float c){ if(x>=c)return 1; if(x<=b)return 0; return (x-b)/(c-b);}
98
99 // Erro (universo +-500)
100 float eNG(float e){return trapEsq(e,-400,-200);} float eNP(float e){return tri(e,-400,-200,0);}
101 float eZ (float e){return tri(e,-100,0,100);} float ePP(float e){return tri(e,0,200,400);}
102 float ePG(float e){return trapDir(e,200,400);}
103 // dErro (universo estreito +-40 -> acao derivativa antecipa frenagem)
104 float dNG(float d){return trapEsq(d,-32,-16);} float dNP(float d){return tri(d,-32,-16,0);}
105 float dZ (float d){return tri(d,-8,0,8);} float dPP(float d){return tri(d,0,16,32);}
106 float dPG(float d){return trapDir(d,16,32);}
107
108 // Base de regras (singletons de saida: NG=-30 NP=-15 Z=0 PP=15 PG=30)
109 const float SAIDA[5][5]={
110     {-30,-30,-30,-15, 0},

```

Linhas 112–131

Base de regras 5x5 (singletons de saída) e função fuzzy(): faz INFERÊNCIA (mínimo) e DEFUZZIFICAÇÃO por centroide.

```

112     {-15,-15, 0, 15, 15},
113     {-15, 0, 15, 15, 30},
114     { 0, 15, 30, 30, 30}};
115
116 float fuzzy(float e,float de){
117     float mE[5]={eNG(e),eNP(e),eZ(e),ePP(e),ePG(e)};
118     float mD[5]={dNG(de),dNP(de),dZ(de),dPP(de),dPG(de)};
119     float num=0,den=0;
120     for(int i=0;i<5;i++){for(int j=0;j<5;j++){
121         float w=min(mE[i],mD[j]); // inferencia (AND = min)
122         if(w>0){ num+=w*SAIDA[i][j]; den+=w; } // centroide (singletons)
123     }
124     return (den==0)?0:num/den;
125 }
126
127 // =====
128 // PLANTA / LEITURAS
129 // =====
130 void calcularPlanta(){ rpm += dt*(K*pwm - B*rpm - C*carga); rpm=constrain(rpm,0,1000); }
131

```

Linhas 133–147

Planta e leitura de carga: o potenciômetro é SEMPRE ativo (carga-base) somado à parcela dos produtos, que decai a cada ciclo.

```

133 // A parcela de produtos decai a cada ciclo (caixas saindo da esteira).
134 void lerCarga(){
135     float base = analogRead(POT_PIN)/4095.0f*40.0f; // potenciometro sempre lido
136     cargaProdutos = max(0.0f, cargaProdutos - DECAIMENTO);
137     carga = constrain(base + cargaProdutos, 0.0f, 40.0f);
138 }
139
140 const float LIMAR=15.0, ALT_SENSOR=20.0; // cm
141 float lerUltrassom(){
142     digitalWrite(TRIG_PIN,LOW); delayMicroseconds(2);

```

```

143 digitalWrite(TRIG_PIN,HIGH); delayMicroseconds(10); digitalWrite(TRIG_PIN,LOW);
144 long dur=pulseIn(ECHO_PIN,HIGH,30000); if(dur==0) return 999;
145 return dur*0.0343/2.0;
146 }
147 // Classifica e registra um produto. Cada caixa SOMA um peso a' carga,

```

Linhas 149–205

Sensoriamento e produção: HC-SR04, registro de produto (que SOMA peso à carga conforme P/M/G), contagem automática (só com esteira em movimento) e manual.

```

149 void classificar(float tam){
150     if(tam<6)         classeTam="Pequeno";
151     else if(tam<13)   classeTam="Medio";
152     else               classeTam="Grande";
153 }
154 void registrarProduto(float tam){
155     contadorProdutos++;
156     ultimoTamanho=tam;
157     classificar(tam);
158     if(classeTam=="Pequeno")   cargaProdutos += PESO_P;
159     else if(classeTam=="Medio") cargaProdutos += PESO_M;
160     else                       cargaProdutos += PESO_G;
161     cargaProdutos = min(cargaProdutos, 40.0f);
162     tProdutos[idxProd%64]=millis(); idxProd++;
163 }
164
165 // (a) Contagem AUTOMATICA por tempo, com tamanho aleatorio P/M/G.
166 //     So conta com a esteira EM MOVIMENTO (rpm>50) -> resolve o caso de
167 //     "continuar contando com carga/velocidade zero".
168 void contagemAutomatica(){
169     if(!modoAuto || emergencia || rpm < 50.0f) return;
170     if(millis())>=tProxAuto{
171         float t[3]={4.0,9.0,16.0};           // alturas exemplo: Peq, Med, Gra
172         registrarProduto(t[random(0,3)]);
173         tProxAuto=millis()+INTERVALO_AUTO;
174     }
175 }
176
177 // (b) Contagem MANUAL pelo HC-SR04 (continua funcionando)
178 void processarProduto(){
179     float dist=lerUltrassom(); bool det=(dist<LIMIAR);
180     if(det && !produtoPresente){           // borda = caixa passando
181         registrarProduto(constrain(ALT_SENSOR-dist,0,ALT_SENSOR));
182         tBloqueioInicio=millis();          // inicia janela de bloqueio
183     }
184     if(!det){ tBloqueioInicio=0; }          // liberou: zera timer de bloqueio
185     produtoPresente=det;
186 }
187 float throughput(){ unsigned long a=millis(); int c=0;
188     for(int i=0;i<min(idxProd,64);i++) if(a-tProdutos[i]<=60000UL) c++; return c; }
189
190 // =====
191 // SUPERVISAO INTELIGENTE (decisao automatica)
192 // Ponto 2: a esteira SO sinaliza travamento se a MESMA caixa ficar
193 // parada em frente ao sensor por >= 4 min. Caixas passando (mesmo do
194 // mesmo tamanho) apenas incrementam a contagem e a esteira segue rodando.
195 // =====
196 void supervisao(){
197     if(produtoPresente && tBloqueioInicio>0 &&
198        (millis()-tBloqueioInicio>=JAM_TIMEOUT_MS) && !emergencia){
199         emergencia=true;
200         client.publish(TOP_ALERTA,"TRAVAMENTO: caixa parada >4 min. Parada automatica.");
201         client.publish(TOP_STATUS,"TRAVAMENTO");
202     }
203 }
204
205 // =====

```

Linhas 207–225

Supervisão inteligente: travamento automático se uma caixa fica parada no sensor além do tempo (4 min por padrão).

```

207 // =====
208 void atuarMotor(){
209     // SERVO (visual no Wokwi): angulo proporcional ao PWM.
210     // Em hardware real, recomenda-se MOTOR CC + motorreductor + driver:
211     // trocar por ledcWrite(canal, map(pwm,0,100,0,255)); // duty -> velocidade
212     int ang = map((int)pwm,0,100,0,180);
213     if(emergencia) ang=0;
214     motor.write(ang);
215 }
216 void atuarLEDs(){
217     if(emergencia){ digitalWrite(LED_VERDE,LOW); digitalWrite(LED_AMARELO,LOW);
218                     digitalWrite(LED_VERMELHO,(millis()/300)%2); return; }
219     float ae=fabs(erro);
220     digitalWrite(LED_VERDE, ae<20?HIGH:LOW);
221     digitalWrite(LED_AMARELO,(ae>=20&&ae<50)?HIGH:LOW);
222     digitalWrite(LED_VERMELHO,ae>=50?HIGH:LOW);
223 }
224 // >>> PONTO buzzer: sirene de dois tons (chama atencao sem ser ruido fixo) <<<
225 void atuarBuzzer(){

```

Linhas 227–260

Atuação: motor (servo/PWM), LEDs por faixa de erro, sirene de dois tons no buzzer e LCD. Botão físico de emergência.

```

227     else          noTone(BUZZER_PIN);
228 }
229
230 void atualizarLCD(){
231     lcd.setCursor(0,0);
232     if(emergencia) lcd.print("*** EMERGENCIA ** ");
233     else { char l[17]; snprintf(l,sizeof(l),"RPM:%4d SP:%4d",(int)rpm,(int)setpoint); lcd.print(l); }
234     lcd.setCursor(0,1);
235     char l2[17]; snprintf(l2,sizeof(l2),"PWM:%3d Prod:%3d",(int)pwm,contadorProdutos); lcd.print(l2);
236 }
237
238 void lerBotao(){
239     static unsigned long t=0;
240     if(digitalRead(BTN_EMERG)==LOW && millis()-t>400){
241         emergencia=!emergencia; t=millis();
242         if(!emergencia) tBloqueioInicio=0;
243         client.publish(TOP_STATUS, emergencia?"EMERGENCIA":"NORMAL");
244     }
245 }
246
247 // =====
248 // MQTT
249 // =====
250 void callback(char* topic, byte* payload, unsigned int len){
251     String m=""; for(unsigned int i=0;i<len;i++) m+=(char)payload[i]; String t=topic;
252     if(t==TOP_CMD_SP){ float v=m.toFloat(); if(v>=0&&v<=1000) setpoint=v; }
253     else if(t==TOP_CMD_LOAD){ float v=m.toFloat(); if(v>=0&&v<=40){ cargaProdutos=v; } }
254     else if(t==TOP_CMD_EMERG){ emergencia=(m.toInt()==1); if(!emergencia) tBloqueioInicio=0;
255     client.publish(TOP_STATUS, emergencia?"EMERGENCIA":"NORMAL"); }
256     else if(t=="esteira/cmd/auto"){ modoAuto=(m.toInt()==1); tProxAuto=millis()+INTERVALO_AUTO; }
257 }
258 void conectarMQTT(){
259     espClient.setInsecure(); client.setServer(mqtt_host,mqtt_port); client.setCallback(callback);
260     while(!client.connected()){

```

Linhas 262–285

MQTT: callback dos comandos (set point, carga, emergência, modo auto), conexão ao broker e publicação de toda a telemetria, incluindo a classe da caixa.

```

262     client.subscribe(TOP_CMD_SP); client.subscribe(TOP_CMD_LOAD); client.subscribe(TOP_CMD_EMERG);
263     client.subscribe("esteira/cmd/auto");
264     client.publish(TOP_STATUS,"NORMAL");
265     } else delay(3000);
266 }
267 }
268 void publicar(){
269     client.publish(TOP_RPM,String(rpm,1).c_str()); client.publish(TOP_PWM,String(pwm,1).c_str());
270     client.publish(TOP_ERRO,String(erro,1).c_str()); client.publish(TOP_CARGA,String(carga,1).c_str());

```

```

271 client.publish(TOP_SP,String(setpoint,1).c_str());
272 client.publish(TOP_PRODUTOS,String(contadorProdutos).c_str());
273 client.publish(TOP_TAMANHO,String(ultimoTamanho,1).c_str());
274 client.publish("esteira/classe",classeTam.c_str());
275 client.publish(TOP_THRU,String(throughput(),0).c_str());
276 }
277
278 // =====
279 void setup(){
280   Serial.begin(115200);
281   pinMode(LED_VERDE,OUTPUT); pinMode(LED_AMARELO,OUTPUT); pinMode(LED_VERMELHO,OUTPUT);
282   pinMode(BUZZER_PIN,OUTPUT); pinMode(TRIG_PIN,OUTPUT); pinMode(ECHO_PIN,INPUT);
283   pinMode(BTN_EMERG,INPUT_PULLUP);
284   motor.attach(MOTOR_PIN);
285   Wire.begin(21,22); lcd.init(); lcd.backlight();

```

Linhas 287–309

setup() e loop(): inicialização e ciclo de 100 ms — lê sensores, roda o fuzzy, atua e publica. É o coração do sistema embarcado.

```

287   WiFi.begin(ssid,password); while(WiFi.status()!=WL_CONNECTED) delay(300);
288   conectarMQTT(); lcd.clear();
289   randomSeed(analogRead(POT_PIN)+millis());
290   tProxAuto=millis()+INTERVALO_AUTO;
291 }
292
293 // =====
294 void loop(){
295   if(!client.connected()) conectarMQTT();
296   client.loop();
297   lerBotao(); lerCarga(); processarProduto(); contagemAutomatica();
298
299   if(emergencia){ pwm=0; rpm=max(0.0f,rpm-30.0f); erro=setpoint-rpm; }
300   else{
301     calcularPlanta();
302     erroAnt=erro; erro=setpoint-rpm;
303     deltaErro=constrain(erro-erroAnt,-40.0f,40.0f);
304     pwm=constrain(pwm+fuzzy(erro,deltaErro),0.0f,100.0f); // FUZZY
305     supervisao();
306   }
307   atuarMotor(); atuarLEDs(); atuarBuzzer(); atualizarLCD(); publicar();
308   delay((int)(dt*1000));
309 }

```